COMMUNITY OVER CODE · GLASGOW 2026

WireGuard for Apache NuttX

A WireGuard VPN that works as a normal network device, `wg0` — on real boards, and inside a NuttX kernel build (on QEMU).

ESP32-S3 & SPRESENSE
TUNNEL UP OVER REAL WI-FI

Real WireGuard peers
LINUX KERNEL, WINDOWS CLIENT

rv-virt knetnsh64
KERNEL BUILD ON QEMU (BUILD_KERNEL)

NuttX 13.0.1 / master / 12.7.0
APPS V0.1.1 · ONE SOURCE

Satoru Akita · Sony Semiconductor Solutions



About Me

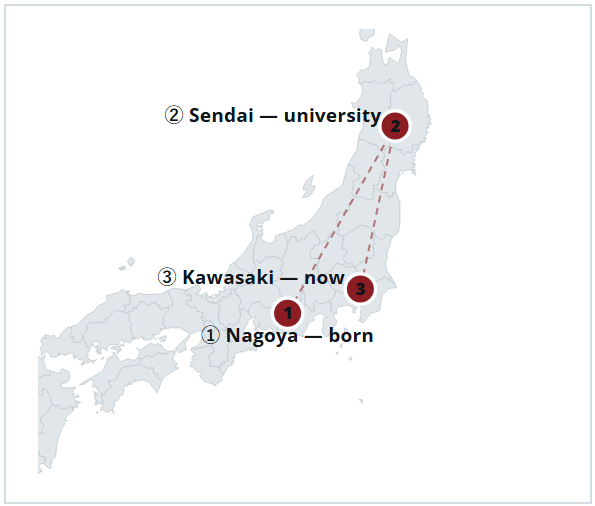
From Japan: I work at Sony, and I build things as a hobby.



Satoru Akita — edge AI engineer at Sony Semiconductor Solutions, working with Apache NuttX from the application side.



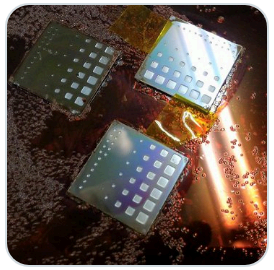
my icon on social media



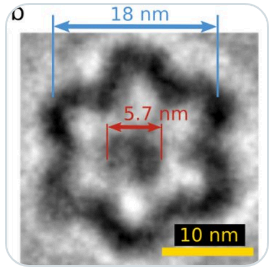
HOBBY ENGINEER



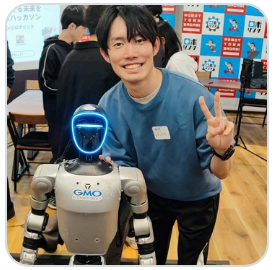
Space



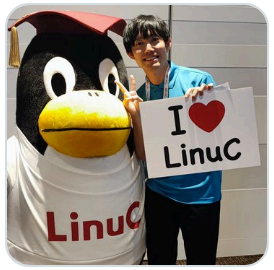
Semiconductors



DNA nanotech



Robots



Linux

🎯 Many hackathons

🎤 Runs software conferences

📖 Wrote a book on open source

Agenda

Seven parts, about thirty minutes.

1. **Overview & background** — ~5 MIN
   What NuttX is · how this started
2. **The problem and the idea** — ~4 MIN
   Reaching a device far away · why WireGuard
3. **The plan** — ~2 MIN
   Reuse wireguard-lwip · fit it into NuttX
4. **Version 1: the app** — ~4 MIN
   First tunnel · six bugs, two in detail · first demo
5. **Version 2: the kernel driver** — ~7 MIN
   Why a driver · a bug in NuttX · testing
6. **Live demo** — ~3 MIN
   StackChan through the tunnel
7. **Summary & status** — ~4 MIN
   Where it stands · what I learned · thanks

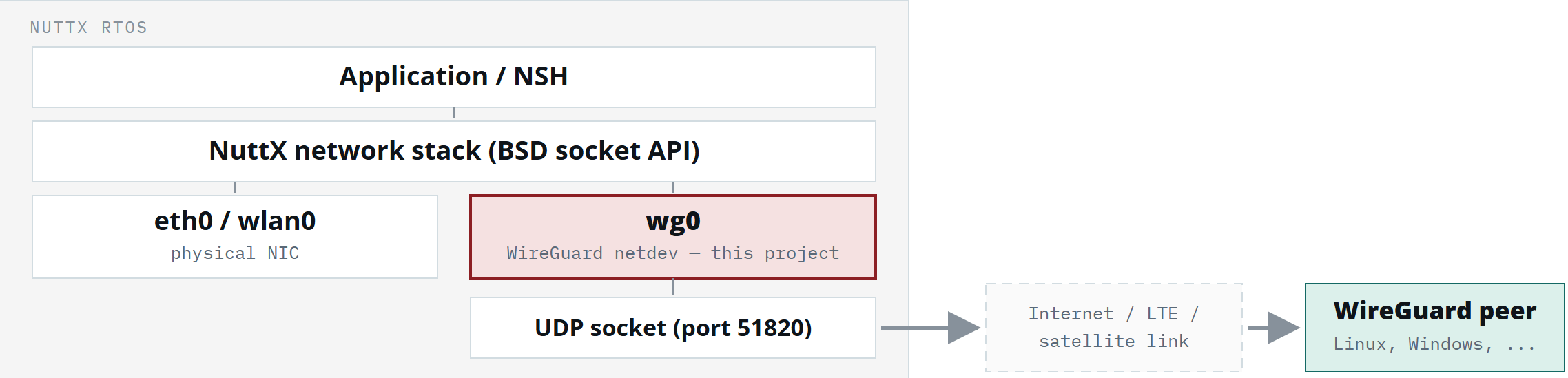
Agenda

Seven parts, about thirty minutes.

1. **Overview & background** — ~5 MIN
   What NuttX is · how this started
2. **The problem and the idea** — ~4 MIN
   Reaching a device far away · why WireGuard
3. **The plan** — ~2 MIN
   Reuse wireguard-lwip · fit it into NuttX
4. **Version 1: the app** — ~4 MIN
   First tunnel · six bugs, two in detail · first demo
5. **Version 2: the kernel driver** — ~7 MIN
   Why a driver · a bug in NuttX · testing
6. **Live demo** — ~3 MIN
   StackChan through the tunnel
7. **Summary & status** — ~4 MIN
   Where it stands · what I learned · thanks

Overview: the project in one slide

This project adds WireGuard to NuttX as a network device.



Packets sent to `wg0` are encrypted and sent out as UDP. UDP that comes in is decrypted and handed over as if it came in on `wg0`. The other side can be any normal WireGuard peer.

TAKEAWAY **Apps see wg0 as a normal network interface. Only the "wire" under it is different: a UDP socket, not a cable.**

How it started

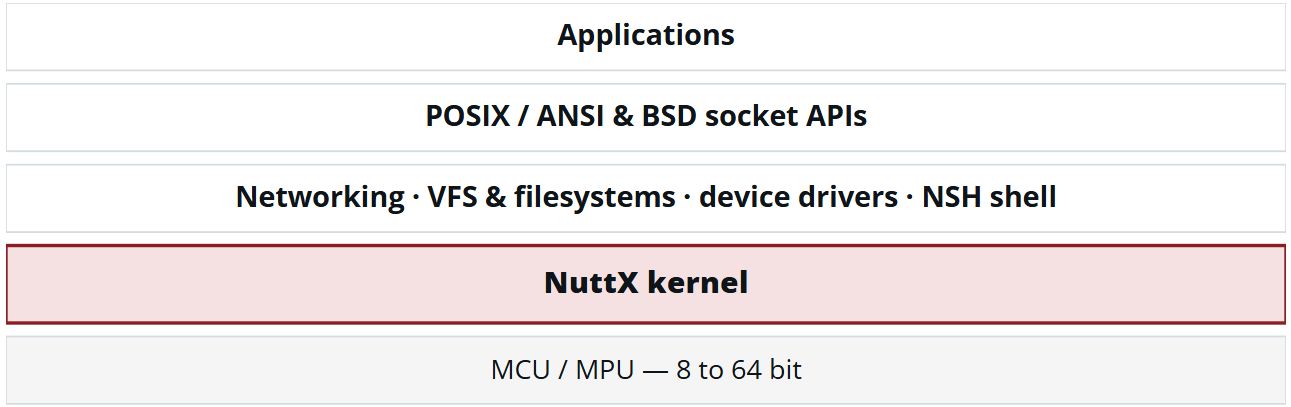
From an idea in February to this talk in October.



TAKEAWAY **The GSoC proposal was not selected, but the work went on. The kernel driver, the harder half, came last.**

Background: the OS behind this work

What is Apache NuttX?



RTOS · POSIX APIs · BSD sockets · Filesystem / VFS · NSH shell · Built-in networking · Device drivers · Apache top-level project

TAKEAWAY **NuttX lets you program small embedded devices the Unix way: POSIX APIs, BSD sockets, files, and devices.**

Background: NuttX and other RTOSes

Where NuttX fits — and where it is used.

RTOS	MAIN FOCUS	TYPICAL USE
FreeRTOS	Small real-time kernel	MCU firmware, control, IoT
Zephyr	All-in-one platform for connected devices	Wearables, industrial & connected IoT
NuttX	**Unix-like embedded OS**	**Networking, file systems, drones, sensing**

> `FreeRTOS/FreeRTOS` 7.8k★ · `zephyrproject-rtos/zephyr` 16.5k★ · `apache/nuttx` 4.1k★
> Stars show how many developers notice a project, not how many devices use it.

ARCHITECTURES
15+ CPU architectures

BOARDS
300+ supported boards

CONFIGS
1,500+ configuration templates

IN PRODUCTION
Sony · PX4 drones · Xiaomi OpenVela (1,000+ SKUs) · Japan's 2024 Moon mission

TAKEAWAY **NuttX has fewer stars than FreeRTOS and Zephyr, but it already runs in real products, from drones to the Moon.**

Sources: nuttx.apache.org · nuttx.apache.org/market-share · docs.px4.io · GitHub star counts, October 2026

History: a long road with NuttX

NuttX at Sony — from audio to edge sensing.

2015 — **Sony audio products**
NuttX inside Sony audio products.

2018–19 — **SPRESENSE / CXD56xx**
Audio, GNSS, camera and multi-core support added to NuttX; shown at Open Source Summit + ELC Europe 2019.

2020 — **Upstream collaboration**
Sony engineers become major NuttX contributors: “A Journey from Fork to Mainline,” NuttX Online Workshop 2020.

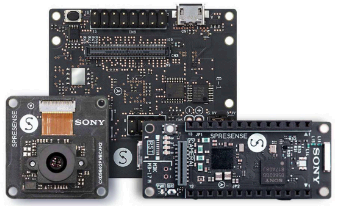
2020s — **Sensing & edge devices**
More camera, GNSS and sensing products.

TAKEAWAY **Sony used NuttX long before edge AI. Years of audio products and SPRESENSE led to sensing and camera systems.**

Sources: M. Ishikawa (Sony), “NuttX for Embedded Linux Developers,” OSS+ELC Europe 2019 · A. Jerpelea (Sony), “A Journey from Fork to Mainline,” NuttX Online Workshop 2020 · nuttx.apache.org/docs (CXD56xx)

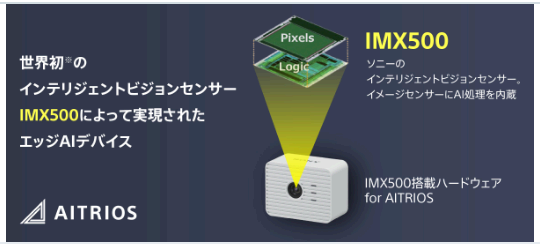
Experience: where I come from

NuttX in SPRESENSE and AITRIOS — my part.



SPRESENSE

NuttX SDK · sensing & camera · GNSS / LTE · prototyping and embedded development.



AITRIOS / edge AI camera

Edge AI · setting up cameras · ESP32 + NuttX · running them on site, not on a desk.

Where I work: application development → system integration → deployment & operation.

TAKEAWAY **My problem was not making NuttX run. It was reaching NuttX devices after they were installed.**

Agenda

Seven parts, about thirty minutes.

1 **Overview & background** ~5 MIN
What NuttX is · how this started

2 **The problem and the idea** ~4 MIN
Reaching a device far away · why WireGuard

3 **The plan** ~2 MIN
Reuse wireguard-lwip · fit it into NuttX

4 **Version 1: the app** ~4 MIN
First tunnel · six bugs, two in detail · first demo

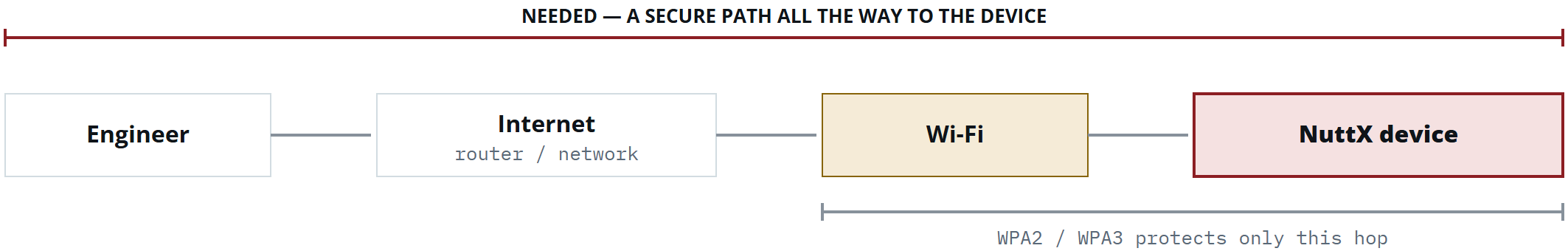
5 **Version 2: the kernel driver** ~7 MIN
Why a driver · a bug in NuttX · testing

6 **Live demo** ~3 MIN
StackChan through the tunnel

7 **Summary & status** ~4 MIN
Where it stands · what I learned · thanks

Problem: Wi-Fi security stops too early

Reaching a device from far away is hard.



OPEN IT TO THE INTERNET

Give the device a public IP and hope.

MAKE YOUR OWN PROTOCOL

Build and maintain your own secure channel.

USE A VENDOR'S CLOUD

Rely on someone else's cloud to reach your own device.

TAKEAWAY **What was missing: a standard, secure path all the way to the NuttX device.**

Options: MQTT, HTTP, router VPN, …

Other ways to reach a device — compared.

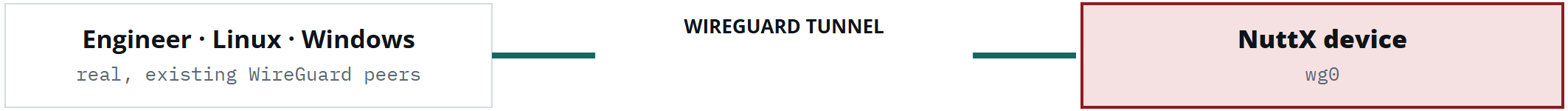
WAY	WHO CONNECTS	ANY SERVICE (TELNET, WEB…)?	SERVER IN THE MIDDLE?	SECURITY ENDS AT	ON NUTTX TODAY
Open a port to the Internet	engineer → device	yes	no	the app (often nothing)	easy, but open to everyone
HTTPS to a cloud API	device → cloud (polling)	no, only your API	yes, a web server	the cloud server	`webclient`, `libcurl4nx` + mbedTLS
MQTT broker (TLS)	device → broker	no, messages only	yes, a broker	the broker	`mqttc`, `paho_mqtt` + mbedTLS
Reverse tunnel / relay (SSH, WebSocket)	device → relay	one port per tunnel	yes, a relay host	the relay host	`dropbear`, `libwebsockets`
VPN on the router	router ↔ office	yes, the whole LAN	no	the router; the LAN hop is open	no change on the device, but you need that router
Carrier closed network (private APN, SIM VPN)	through the carrier	yes	yes, the carrier	the carrier gateway	LTE modem; cellular only
VPN on the device (this talk)	**either side**	**yes, any IP service**	**no, any WireGuard peer**	**the device itself**	`wg0`

TAKEAWAY **MQTT and HTTPS are great for data. To log in and fix a device, end to end, with no server in the middle, the VPN has to be on the device.**

Mesh VPN agents (Tailscale, ZeroTier) are another “VPN on the device” option, but they usually need a bigger OS. NuttX clients: nuttx-apps 13.0.1 `netutils/` (mqttc, paho_mqtt, webclient, libcurl4nx, libwebsockets, dropbear) and `crypto/mbedtls`.

Solution: a VPN on the device

Why WireGuard is a good fit.

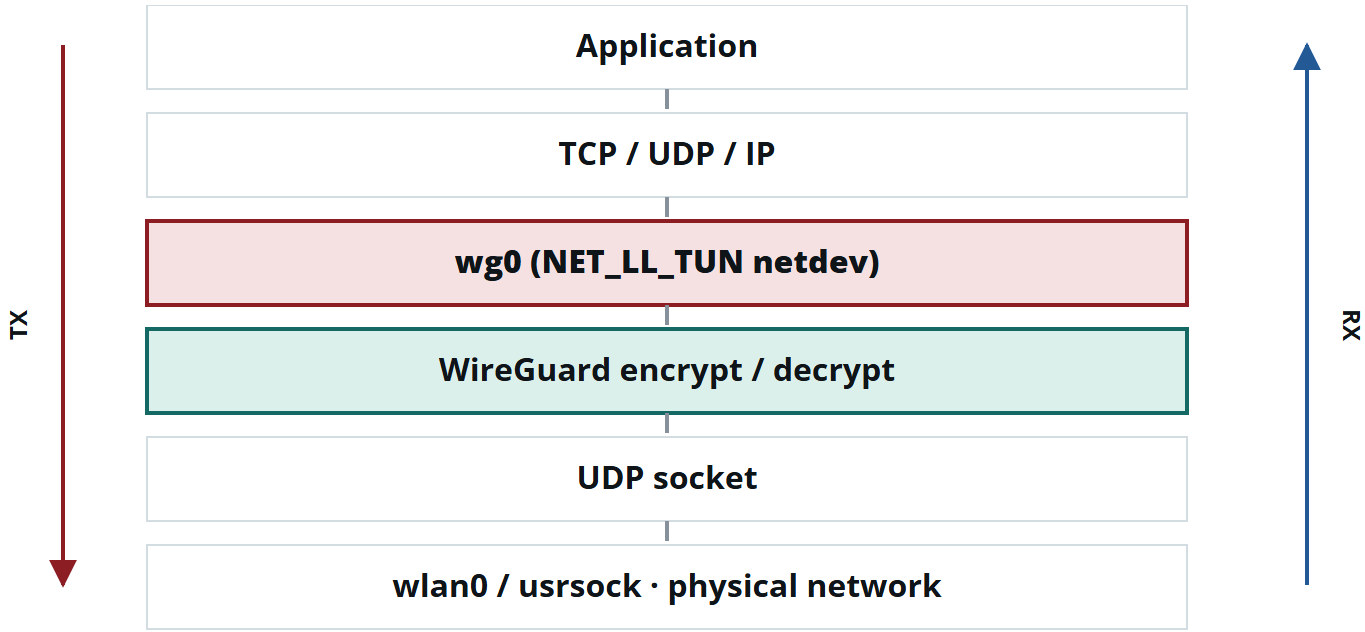


- **Modern IP-layer VPN**
  small and simple: much less code than IPsec or OpenVPN.
- **UDP transport, public-key peers**
  no certificates and no session setup to manage.
- **Already everywhere**
  in the Linux kernel, with an official Windows client. It works with existing peers, not a custom protocol.

TAKEAWAY **WireGuard solves the access problem with a standard protocol, not a homemade security layer or a vendor cloud.**

Architecture: where wg0 sits

WireGuard on the NuttX network stack.



TAKEAWAY **To NuttX, wg0 is a normal network device. Under it, WireGuard turns its packets into encrypted UDP.**

Agenda

Seven parts, about thirty minutes.

1 **Overview & background** ~5 MIN
What NuttX is · how this started

2 **The problem and the idea** ~4 MIN
Reaching a device far away · why WireGuard

3 **The plan** ~2 MIN
Reuse wireguard-lwip · fit it into NuttX

4 **Version 1: the app** ~4 MIN
First tunnel · six bugs, two in detail · first demo

5 **Version 2: the kernel driver** ~7 MIN
Why a driver · a bug in NuttX · testing

6 **Live demo** ~3 MIN
StackChan through the tunnel

7 **Summary & status** ~4 MIN
Where it stands · what I learned · thanks

Porting: four functions to write

Reusing wireguard-lwip without lwIP.

PROTOCOL & CRYPTO — KEEP

Handshake, transport, BLAKE2s, ChaCha20-Poly1305, Curve25519. Portable C, no OS dependency. Crypto used as-is; the protocol core got two small, documented fixes.

PLATFORM HOOKS — ADAPT

Everything OS-specific sits behind `wireguard-platform.h` — four functions: clock, random bytes, TAI64N time, load check.

LWIP NETWORK GLUE — REPLACE

`wireguardif.c`'s lwIP `netif` registration and buffers. NuttX has no lwIP, so this layer is the real porting work.

TAKEAWAY **A clean line between the protocol and the OS let the crypto move unchanged, and the protocol core with only two small fixes. Only the OS and network glue had to change.**

Design: one virtual network card

Mapping lwIP onto NuttX netdev.

CONCEPT	LWIP	NUTTX
Network device	struct netif	**struct net_driver_s**
Buffers	pbuf	**iob**
Registration	netif_add()	**netdev_register()**
Start sending	callback	**devif_poll()**

> **`wg0` is a `NET_LL_TUN` netdev, and its wire is a UDP socket.**

TAKEAWAY **Porting meant using NuttX's own building blocks, not bringing lwIP into NuttX.**

Agenda

Seven parts, about thirty minutes.

1. **Overview & background** — ~5 MIN
   What NuttX is · how this started
2. **The problem and the idea** — ~4 MIN
   Reaching a device far away · why WireGuard
3. **The plan** — ~2 MIN
   Reuse wireguard-lwip · fit it into NuttX
4. **Version 1: the app** — ~4 MIN
   First tunnel · six bugs, two in detail · first demo
5. **Version 2: the kernel driver** — ~7 MIN
   Why a driver · a bug in NuttX · testing
6. **Live demo** — ~3 MIN
   StackChan through the tunnel
7. **Summary & status** — ~4 MIN
   Where it stands · what I learned · thanks

Result: the first working tunnel

A working tunnel is only the beginning.

LAYER	WORK NEEDED	IN THIS PORT
Protocol & crypto	Almost none — portable C	**crypto byte-identical; protocol core: two small fixes (PATCHES.md)**
OS hooks	Small — four functions	**clock, /dev/urandom, TAI64N, load**
Network glue	The real work	**netdev + UDP socket + RX task**

It worked: `wg genkey / set / setconf` at run time. No rebuild to change keys or peers.

And this is where the interesting part starts.

TAKEAWAY **Getting a tunnel up was the easy step. Almost everything interesting came after it already "worked."**

Where it was hard: six bugs

The tunnel worked fast. Six bugs took most of the time.

#	BUG	SHOWED UP IN	LESSON
1	The receive thread stopped when the `wg` command exited	apps version	`pthread_detach()` frees memory later. It does not keep a thread alive.
2	**ping works, TCP dies** `in detail`	**apps version**	**A file descriptor belongs to one task group.**
3	**“No real impact” was 10×** `in detail`	**apps version**	**Measure a quick fix before you call it free.**
4	Stack 94% full; saving the config froze the board	real boards only, never sim	Measure on real hardware. Sim's big stacks hide it.
5	Handshake OK, data fails: NuttX's own ChaCha20-Poly1305 `part 5`	kernel driver	A passing handshake does not prove the data path.
6	A 6 KB array on a 3 KB kernel stack `part 5`	BUILD_KERNEL (QEMU)	The same stack lesson, one level deeper.



For implementation details, debugging notes, and test results, see the GitHub repository.
`github.com/wwlapaki310/gsoc2026-nuttx-wireguard`

TAKEAWAY **Most of them hid behind a test that passed, so I first looked in the wrong place.**

Bug 2 / 6 · my favorite

ping works. TCP dies.

Same tunnel: `ping` is **0% loss.** A telnet 3-way handshake even **completes**, then the connection dies.

```
wg_encrypt_and_send: psock_sendto
ret=-1 errno=9  EBADF
```

The sends that **worked** ran inside the `wg_rx` task. Sends from another task got EBADF.

Cause: an fd belongs to one **task group**; it is not global.

Fix: use the internal API without fds: `psock_*()` on a `struct socket`, which any task can use.

TAKEAWAY **If only some tasks fail, look at what each task owns. Here, it was the file descriptor.**

Bug 3 / 6

“No real impact” was 10×.

Quick fix: check the socket every 50 ms.

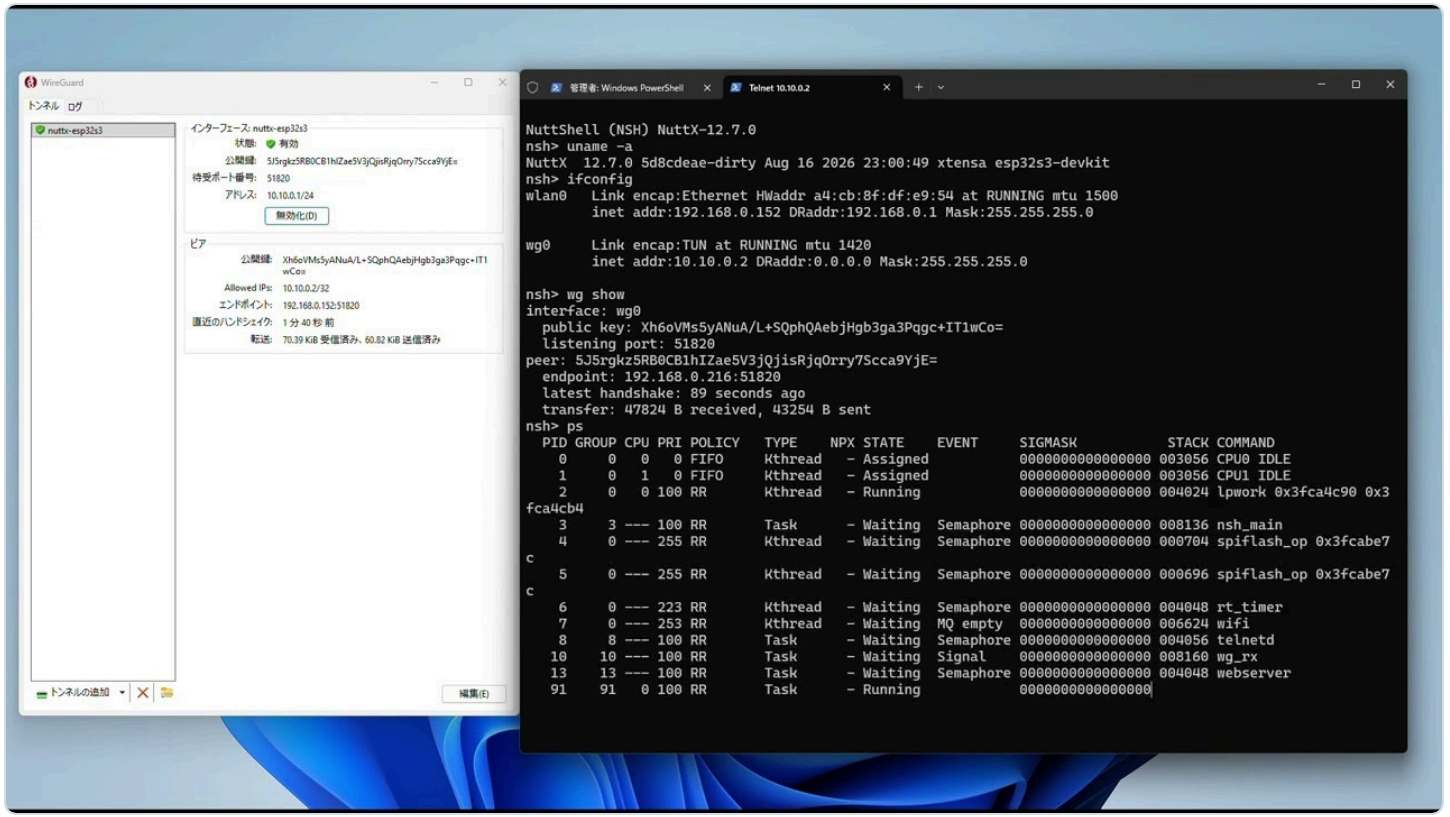
The fix without fds also broke `poll()`. I thought the only cost was waking an idle CPU. Wrong: each packet waited up to 50 ms, and that made it **about 10× slower.**

Real fix: a callback on `psock_poll()` and a semaphore, a real blocking wait.

TAKEAWAY **A quick fix you never measure can look free. Measure it.**

Version 1 demo: the first video

Version one works: telnet and a web page through the tunnel.




```
1  telnet 10.10.0.2
2  uname -a, ifconfig
3  webserver &
4  browser: http://10.10.0.2/
```

ESP32-S3 DevKit, apps version (FLAT). USB unplugged.

Other end: the official Windows WireGuard client.

Video: youtu.be/1kyX2av5WG4

TAKEAWAY **Version one was enough for a demo. Next, I made it a real NuttX driver.**

Agenda

Seven parts, about thirty minutes.

1. **Overview & background** — ~5 MIN
   What NuttX is · how this started
2. **The problem and the idea** — ~4 MIN
   Reaching a device far away · why WireGuard
3. **The plan** — ~2 MIN
   Reuse wireguard-lwip · fit it into NuttX
4. **Version 1: the app** — ~4 MIN
   First tunnel · six bugs, two in detail · first demo
5. **Version 2: the kernel driver** — ~7 MIN
   Why a driver · a bug in NuttX · testing
6. **Live demo** — ~3 MIN
   StackChan through the tunnel
7. **Summary & status** — ~4 MIN
   Where it stands · what I learned · thanks

Why the kernel: from app to driver

Version 1 proved the protocol. Version 2 makes it a real NuttX device.

Version 1: an app

Lives in `apps/` and reaches into kernel internals (`psock_*`).

Works only in a **FLAT** build. It can never be a proper in-tree driver.

Version 2: a kernel driver

`wg0` lives in `drivers/net/wireguard/`. A small `wg` command sets it up through **ioctl**.

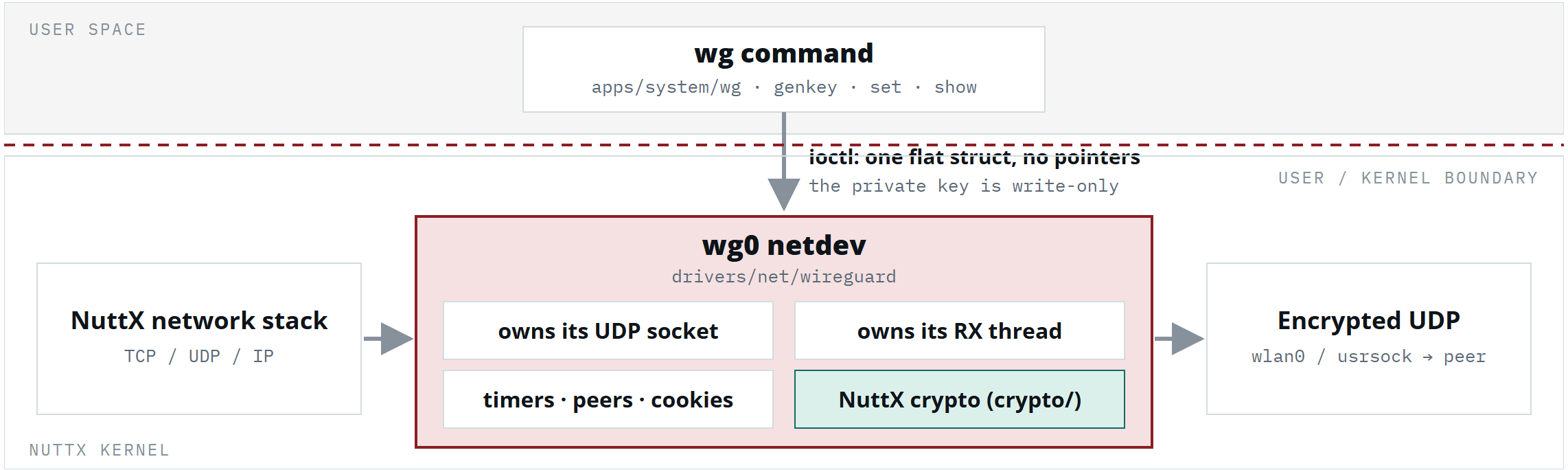
Made for **FLAT, PROTECTED and KERNEL** builds.

> **Build modes.** FLAT: apps and kernel share one memory space. PROTECTED: the MPU keeps apps out of kernel memory. KERNEL: each program has its own address space (MMU).

TAKEAWAY **The goal: one device that fits how NuttX is built, in every build mode, in a shape the community can review.**

The kernel design: the big picture

One kernel device, set up by a small command.



TAKEAWAY **The driver owns everything that must keep running. The command only sends settings, and then it can exit.**

Bugs 5 & 6 / 6: found in the kernel part

Two more bugs, and one was in NuttX itself.

In NuttX's own crypto

`chacha20poly1305` put the counter in the wrong nonce bytes. Counter 0 still matches, so **the handshake works, and data packets after the first fail.**

Nothing in NuttX had used it before. Two-line fix, kept as its own `crypto:` change.

Only in a KERNEL build

Setting the peers put a **6 KB array** on a **3 KB kernel stack**: overflow, then panic.

Sim (FLAT, big stacks) never hit it. Moved to the heap.

> **Also found in NuttX:** SPRESENSE (cxd56) did not boot on NuttX 13.0.1 and master with `CONFIG_RTC_HIRES`. Fixed locally; the report is drafted, not sent.

TAKEAWAY **Porting into NuttX also found bugs in NuttX. A port can give something back to the project.**

Testing: what ran where

Real boards in FLAT builds. KERNEL and PROTECTED on QEMU.

FLAT · REAL BOARDS

ESP32-S3 **2026-10-12, exact submitted series:** handshake with Linux, ping both ways, full MTU.

ESP32-S3 + SPRESENSE earlier build: tunnel to the Windows client.

SPRESENSE 40 down/up: memory the same. RX thread: 1472 B of 6096.

KERNEL · QEMU RV-VIRT

Tunnel to Linux WireGuard. `wg` is a separate program that calls in through syscalls.

4 CPUS heavy traffic while changing settings and doing down/up: no fault (one scripted run).

PROTECTED · QEMU RV-VIRT

Tunnel works. Every ioctl crosses the MPU boundary.

A rare fault: the `wg` stack overflowed, because a syscall runs on the caller's stack.

Fixed with a bigger default: **20/20**.

Not shown yet: PROTECTED or KERNEL on real hardware.

Tried PROTECTED on the CoreS3, but the board does not boot any bootloader-based image (not a WireGuard problem).

For implementation details, debugging notes, and test results, see the GitHub repository.
`github.com/wwlapaki310/gsoc2026-nuttx-wireguard`

Testing: what ping misses

Three problems that ping would never notice.

SAME KEY ON EVERY BOOT?

`SPRESENSE` 3 resets, **3 different keys**.

A quick check, not a proof of good randomness.

KEYS LEFT IN MEMORY AFTER DOWN?

`SIM` session keys **all zero** after `down`.

The static key stays, on purpose, so `up` works again.

A LEAK THAT SHOWS AFTER HOURS?

`SIM` 151 rekeys, 100 down/up: **nothing lost**.

Memory back to the start-up count. Not a week-long run.

TAKEAWAY **Each risk needs its own test. A passing tunnel is not one of them.**

Design choice: trust the clock?

The timestamp problem has no easy answer.

Each handshake needs a newer timestamp than the peer has seen, **even after a reboot**.

Chosen

Real-time clock + the highest value used this boot.
Same rule as Linux and wireguard-go, **plus a warning** when the clock was never set.

Not chosen

Save every timestamp: wears out flash.
Reserve a range in flash: needs a write guarantee I could not verify.
Wait: as long as it last ran.

Same board, nothing sent to it: clock unset → no handshake in 75 s. Clock set → 4.1 s.

TAKEAWAY **Without a clock, a device may not reconnect by itself, and a ping test will not show it.**

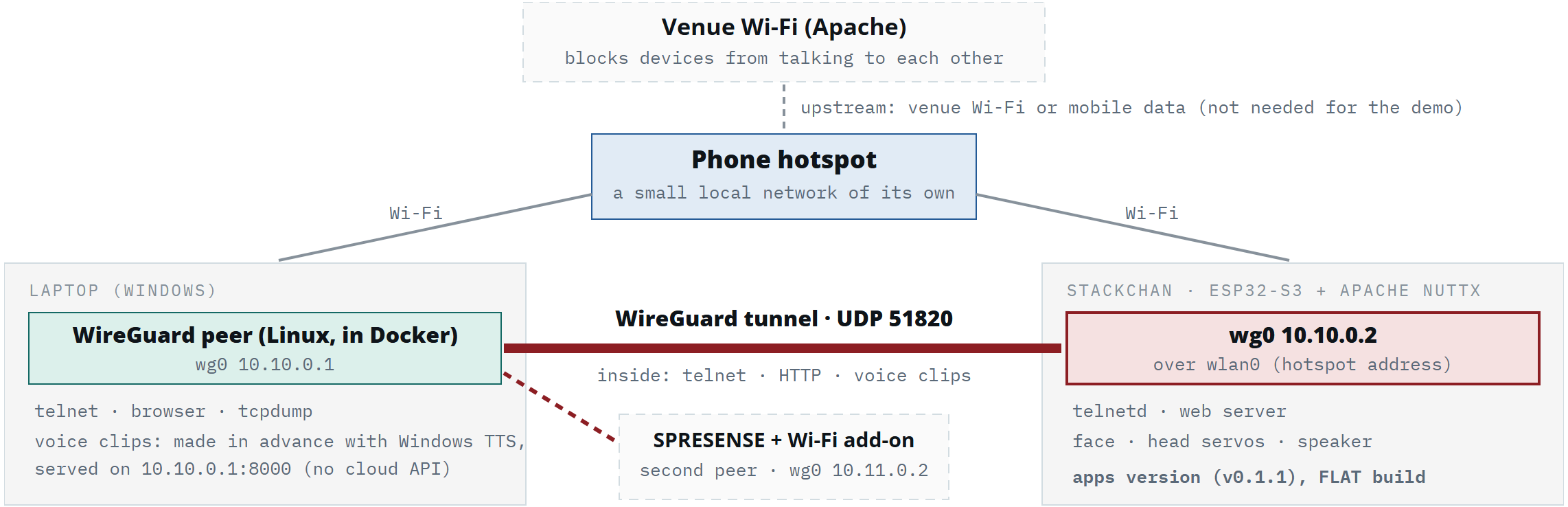
Agenda

Seven parts, about thirty minutes.

1. **Overview & background** — ~5 MIN
   What NuttX is · how this started
2. **The problem and the idea** — ~4 MIN
   Reaching a device far away · why WireGuard
3. **The plan** — ~2 MIN
   Reuse wireguard-lwip · fit it into NuttX
4. **Version 1: the app** — ~4 MIN
   First tunnel · six bugs, two in detail · first demo
5. **Version 2: the kernel driver** — ~7 MIN
   Why a driver · a bug in NuttX · testing
6. **Live demo** — ~3 MIN
   StackChan through the tunnel
7. **Summary & status** — ~4 MIN
   Where it stands · what I learned · thanks

Demo setup: the network

One tunnel, on a phone's local network.



TAKEAWAY **The phone and the venue only ever see encrypted UDP, with no server in the middle and no cloud.**

Demo: live on the StackChan

Log in through the tunnel, and talk to the robot.

```
wg show           # handshake, bytes
telnet 10.10.0.2   # log in
ifconfig          # wlan0 and wg0
stackchan face happy
stackchan say hello.wav
tcpdump: outside vs. inside
wg show           # bytes went up
```

Outside the tunnel (the phone's network): only encrypted UDP on port 51820.

Inside the tunnel (wg0): the telnet text you just typed.

Runs the apps version (v0.1.1), FLAT build.

Backup video (earlier demo): youtu.be/1kyX2av5WG4

Agenda

Seven parts, about thirty minutes.

1. **Overview & background** — ~5 MIN
   What NuttX is · how this started
2. **The problem and the idea** — ~4 MIN
   Reaching a device far away · why WireGuard
3. **The plan** — ~2 MIN
   Reuse wireguard-lwip · fit it into NuttX
4. **Version 1: the app** — ~4 MIN
   First tunnel · six bugs, two in detail · first demo
5. **Version 2: the kernel driver** — ~7 MIN
   Why a driver · a bug in NuttX · testing
6. **Live demo** — ~3 MIN
   StackChan through the tunnel
7. **Summary & status** — ~4 MIN
   Where it stands · what I learned · thanks

Giving back: where it stands

The Apache Way: small steps, in the open.

- **A small `crypto:` fix**

  the ChaCha20-Poly1305 nonce fix, kept apart from the driver because it is useful on its own.

- **The driver**

  `drivers/net/wireguard`: the kernel device, ABI, and docs, plus two small network-core changes.

- **The command**

  `apps/system/wg`: talks to the driver. Its stack default now covers PROTECTED builds.

- **Checked so far**

  checkpatch / nxstyle pass, each commit builds on its own, licenses listed. The series runs on the ESP32-S3 (FLAT, 2026-10-12).

TAKEAWAY **Small, separate pieces are easier to review, and the crypto fix is useful even without the driver.**

Status: prepared as small PRs. Not signed off yet; nothing sent upstream.

What I learned

Three things this project taught me.

LESSON 1

From application user to OS contributor

For years I used NuttX from the application side.

This project took me into drivers, the kernel, and bugs in NuttX itself.

LESSON 2

AI helped me code, but understanding was my job

I used Claude a lot. Research and code went much faster.

OS internals were new to me. I did not take AI code as it was. I asked, read, and tested until I could explain it myself.

That was the hard part.

LESSON 3

A welcoming open-source community

Good docs and contributing guides. Active discussion and review.

Open to AI tools, with people responsible: `Assisted-by`, and AI never adds `Signed-off-by`.

CI and hackathons help new people join.

HOPE **I hope to keep contributing to Apache NuttX and the open-source community, and I look forward to seeing NuttX continue to grow.**

Summary

What I built, and three technical lessons.

BUILT `wg0`: **a WireGuard network device in the NuttX kernel. It runs on ESP32-S3 and SPRESENSE (FLAT) and in KERNEL / PROTECTED builds on QEMU, with standard WireGuard peers. Prepared as small PRs; nothing sent yet.**

– Test beyond the handshake

long data runs, real hardware, every build mode.

– A clean OS boundary pays off

the apps version: four CPU types, three NuttX versions, no code changes.

– Porting to a project can make the project better

a hidden bug in NuttX's own ChaCha20-Poly1305 was found on the way, and the fix stands on its own.

Thanks: the people behind this talk

Many people helped me get here.

Alan

for telling me about this conference and encouraging me to submit a talk.

The Apache NuttX community

for talking with me about my GSoC proposal, and for encouraging me to keep going.

The Apache Software Foundation

and the Community Over Code organizers and volunteers, for making this event happen.

Sony's NuttX developers, past and present

who brought NuttX into our products and gave so much back to the project.

COMMUNITY OVER CODE · GLASGOW 2026

Thank you

Questions welcome — the kernel driver, the bugs, the ioctl ABI, or the hardware.

youtu.be/1kyX2av5WG4
DEMO

github.com/wwlapaki310/gsoc2026-nuttx-wireguard
CODE & RECORDS

Satoru Akita · Sony Semiconductor Solutions

